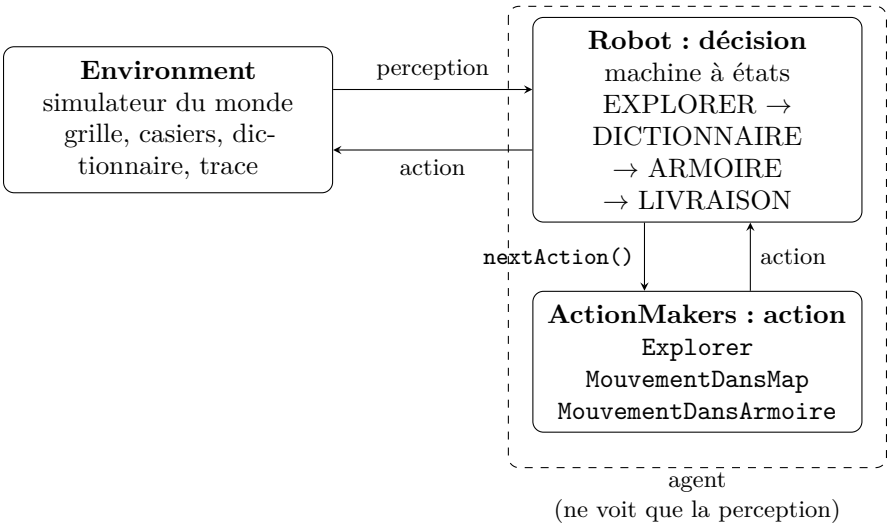
Le robot de réconfort — rapport Partie 1 (HAI716)

Fauron, Lamkaddem, Thomas

1. Architecture



— **Perception.** `Environment.get_perception` renvoie au robot un paquet d'informations qu'il a le droit de lire : sa position, les quatre cases adjacentes, la position du sélecteur, le contenu du casier courant et le résultat de sa lecture du dictionnaire. Le robot ne lit jamais la grille du fichier de carte.

— **Mémoire.** L'`Environment` stocke la totalité des informations qui ne relèvent pas de la réflexion ; l'agent les redemande à chaque tour pour mettre à jour son état et proposer sa prochaine action. Seules les informations issues de sa réflexion, comme la carte mentale (`carte_mental`), sont stockées côté agent et partagées avec les objets `ActionMaker`.

— **Décision.** `Robot.updateState`, `Robot.nextState` et `Robot.compute_NextMove` : le robot calcule d'abord son état à partir des nouvelles informations du tour, puis choisit une action selon cet état.

— **Action.** Les `ActionMaker` produisent les actions de l'agent ; l'`Environment` les récupère avec `get_agentsActions`, puis `_handleAction` les exécute et les écrit dans la trace.

Justification. La frontière `Environment`/agent est la frontière de l'information : tout ce qui est non perçu reste du côté du monde. De plus, c'est le monde qui conserve le véritable « corps » de l'agent (position et inventaire). C'est pour cela que nous avons choisi de lui communiquer ces informations dans la perception, afin de respecter cette logique. Il est théoriquement possible de tricher en accédant à `grid` ou à `casiers` depuis le code de l'agent, mais cela n'est pas fait. La frontière décision/action permet de changer la stratégie de déplacement sans toucher aux états, tout en gardant une clarté dans le code.

2. La stratégie de replanification

Lorsqu'il découvre un mur ou une case inaccessible, le robot actualise sa carte mentale à partir de ses perceptions. Il recalcule ensuite un chemin vers les cases à explorer ou sa destination à l'aide d'un parcours en largeur. Il ne répare donc pas son ancien chemin : il effectue un nouveau calcul depuis sa position actuelle. Cette méthode garantit un chemin optimal sur la carte connue, avec un coût de $O(V + E)$ par recherche, où V représente le nombre de cases et E le nombre de connexions entre elles.

Gestion des déplacements et exploration.

La gestion du déplacement se fait en deux grandes étapes. Dans un premier temps, le robot se charge d'explorer l'appartement afin de créer sa carte mentale, en suivant une approche semblable à un parcours en profondeur. Dans un second temps, il utilise cette carte mentale pour calculer le meilleur chemin entre deux positions à l'aide d'un parcours en largeur.

La phase d'exploration repose principalement sur une structure de pile. À chaque étape, le robot récupère sa perception de l'environnement et examine les cases voisines de sa position actuelle. Les

cases qui sont accessibles et qui n'ont pas encore été visitées sont alors ajoutées à la pile. Le robot dépile ensuite une position et se déplace vers celle-ci afin de poursuivre son exploration. Ce fonctionnement permet de progresser dans l'appartement en explorant les différentes possibilités de déplacement, tout en mémorisant les cases déjà visitées et les obstacles rencontrés. Au fur et à mesure de ses déplacements, sa carte mentale est donc actualisée à partir des informations récupérées par ses perceptions.

La phase de recherche de chemin repose quant à elle sur un parcours en largeur, qui permet de trouver un chemin optimal entre deux positions sur la carte mentale connue du robot. Pour cela, le parcours utilise une structure d'arbre afin de pouvoir reconstruire le chemin une fois la position but atteinte. Chaque nouveau voisin exploré est associé à un nœud dont le père correspond à la position depuis laquelle il a été découvert. Lorsque la position but est trouvée, le robot remonte alors de nœud en nœud, en suivant les liens vers les pères, jusqu'à retrouver la position de départ. Cette remontée permet de construire une liste ordonnée de positions correspondant au chemin à suivre.

Ce calcul est répété à chaque étape du déplacement : le robot recalcule donc son chemin depuis sa position actuelle plutôt que de suivre un itinéraire calculé une seule fois à l'avance. Cela lui permet de prendre en compte les nouvelles informations obtenues au cours de ses déplacements, notamment lorsqu'il découvre un obstacle qui n'était pas encore présent dans sa carte mentale. Il peut ainsi adapter son déplacement en fonction de l'environnement qu'il découvre.

3. Critique de la règle de repli

La règle de repli actuelle est discutable. Elle présente l'avantage d'être simple à mettre en œuvre : lorsqu'aucun objet correspondant à l'émotion demandée n'est disponible, le robot cherche dans les émotions voisines sur la roue de Plutchik. Cependant, cette proximité est uniquement basée sur la représentation circulaire des émotions et ne garantit pas que les émotions soient réellement compatibles. Par exemple, répondre à une personne triste avec un objet associé au dégoût peut sembler inadapté, voire contre-productif. Une proximité sur la roue ne signifie donc pas nécessairement qu'un objet sera pertinent pour la personne.

Une meilleure solution serait de définir explicitement des relations de compatibilité entre les émotions, en privilégiant les objets ayant une fonction réconfortante adaptée à l'émotion exprimée. Par exemple, pour une demande liée à la tristesse, le robot pourrait privilégier des objets associés au réconfort, au soutien ou à l'apaisement, plutôt que de sélectionner automatiquement une émotion voisine. Il serait également intéressant de prévoir un objet neutre ou une solution de secours lorsqu'aucun objet suffisamment adapté n'est disponible.

4. Trois échecs de notre robot

Échec 1 : « armoire vide » déclaré sans regarder le casier. Si le sélecteur est déjà sur le meilleur casier candidat en entrant dans l'état ARMOIRE, le robot envoie `PRENDRE` sans objet et déclare l'armoire vide. `MouvementDansArmoire` est créé après le `updateState` du pas, donc son premier `nextAction` s'exécute avant d'avoir lu `contenu_casier`. Constaté dans `scenario_05` (demande 2) : le casier `[1,4]`, déjà vidé, n'est pas suivi du candidat suivant, et la demande échoue alors que l'armoire contient d'autres objets.

Échec 2 : l'exploration exhaustive avant toute livraison. Le robot n'a le droit de livrer qu'après avoir exploré toute la carte, même quand le dictionnaire et l'armoire sont connus et proches.

Scénario	Pas avant la 1re consultation	Pas au total	Part
01	69	110	63 %
02	140	331	42 %
03	151	407	37 %
04	198	404	49 %
05	226	476	48 %

La colonne « Pas avant la première consultation » inclut l'exploration et le trajet jusqu'au dictionnaire. Cause : l'état initial EXPLORER ne se termine que lorsque la frontière est vide ; c'est nécessaire

pour prouver qu'un résident est inaccessible (annexe B), mais pas pour livrer un résident accessible. Correction : explorer à la demande, jusqu'à connaître un chemin vers la cible, et ne poursuivre l'exploration que pour prouver une inaccessibilité.

Échec 3 : changer de demande coûte un pas au lieu de zéro. Un échec connu d'avance ne devrait rien coûter, mais dans certains scénarios, tels que notre version modifiée de `scenario_01`, chacune des deux demandes adressées à `R1`, qui est entouré de murs, se solde par un `ATTENDRE` suivi de la récupération de la prochaine demande.